Operating System

Module – 3

Deadlock Detection

Deadlock Detection

- An algorithm that examines the state of the system to determine whether a deadlock has occurred.
- An algorithm to recover from the deadlock

Single Instance of Each Resource Type

- If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a **wait-for** graph.

Deadlock Detection

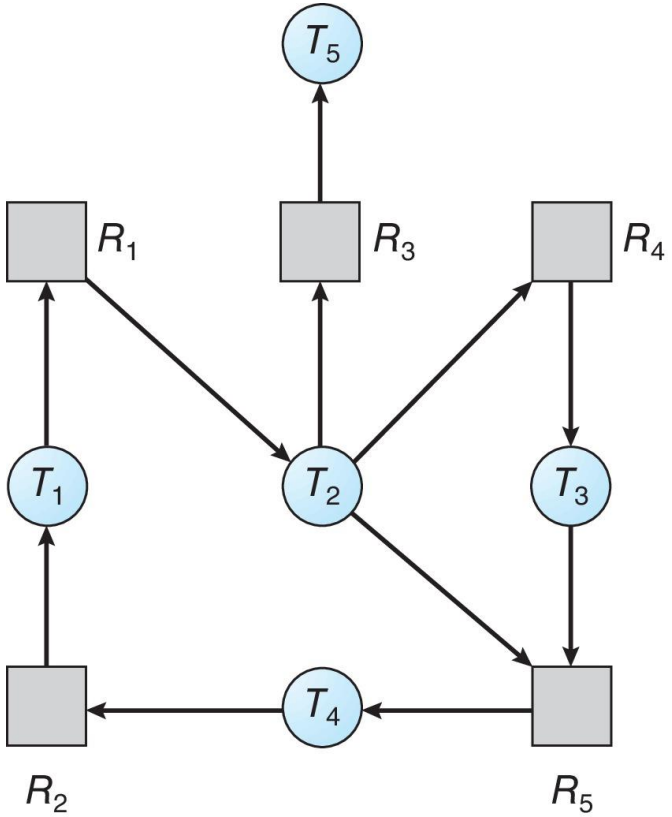
A deadlock exists in the system if and only if the wait-for graph contains a cycle.

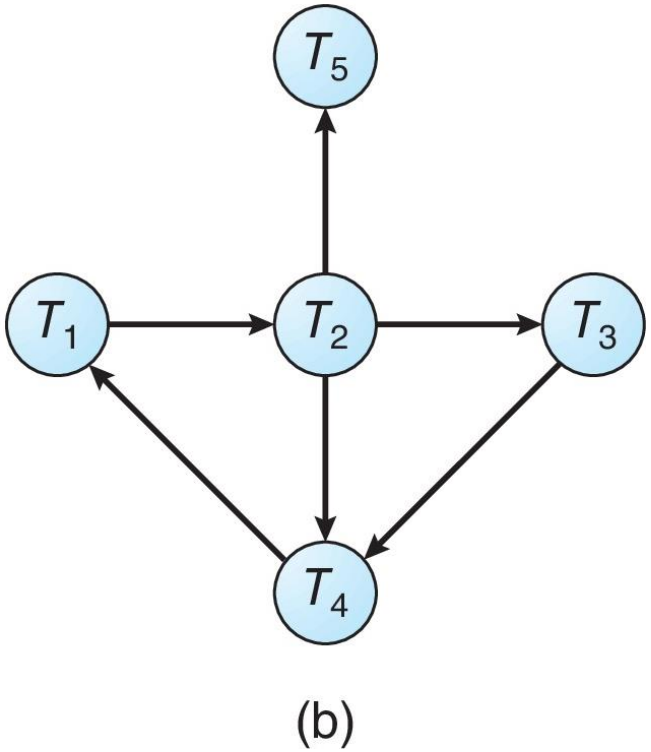
To detect deadlocks, the system needs to maintain the wait for graph and periodically invoke an algorithm that searches for a cycle in the graph.

An algorithm to detect a cycle in a graph requires **$O(n^2)$ operations**, where n is the number of vertices in the graph.

Deadlock – Management - Detection

For the given resource allocation Graph create wait for graph,





(b)

Deadlock Detection

Several Instances of a Resource Type

Data structures used

- **Available**. A vector of length *m* indicates the *number of available resources of each type*.
- **Allocation**. An *n* × *m* matrix defines the *number of resources of each type currently allocated to each thread*.
- **Request**. An *n* × *m* matrix indicates *the current request of each thread*.

Deadlock Detection

1. Let *Work* and *Finish* be vectors of length m and n, respectively. Initialize $Work = Available$. For $i = 0, 1, ..., n-1$, if $Allocation_i \neq 0$, then $Finish[i] = false$. Otherwise, $Finish[i] = true$.
2. Find an index i such that both
   a. $Finish[i] == false$
   b. $Request_i \leq Work$

   If no such i exists, go to step 4.
3. $Work = Work + Allocation_i$
   $Finish[i] = true$
   Go to step 2.
4. If $Finish[i] == false$ for some i, $0 \leq i < n$, then the system is in a deadlocked state. Moreover, if $Finish[i] == false$, then thread T_i is deadlocked.

Deadlock Detection

The following snapshot represents the current state of the system:

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
T_0	0 1 0	0 0 0	0 0 0
T_1	2 0 0	2 0 2	
T_2	3 0 3	0 0 0	
T_3	2 1 1	1 0 0	
T_4	0 0 2	0 0 2	

Deadlock Detection

The following snapshot represents the current state of the system:

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
T_0	0 1 0	0 0 0	0 0 0
T_1	2 0 0	2 0 2	
T_2	3 0 3	0 0 0	
T_3	2 1 1	1 0 0	
T_4	0 0 2	0 0 2	

<*T*0, *T*2, *T*3, *T*1, *T*4>

Detection-Algorithm Usage

When should we invoke the detection algorithm? It depends on two factors:

1. How ***often*** is a deadlock likely to occur?

2. How ***many*** threads will be affected by deadlock when it happens?

Detection-Algorithm Usage

- If deadlocks occur frequently, then the detection algorithm should be invoked frequently
- Invoke the deadlock detection algorithm every time a request for allocation cannot be granted – *it will incur considerable overhead in computation time*
- Invoke the algorithm at defined intervals

Deadlock Recovery

1. Process Termination

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- *In which order should we choose to abort?*
  - Priority of the thread
  - How long has the thread computed, and how much longer to completion
  - Resources that the thread has used
  - Resources that the thread needs to complete
  - How many threads will need to be terminated
  - Is the thread interactive or batch?

Deadlock Recovery

2. Resource Preemption

- Selecting a victim – *resources and process*
- Rollback – return to some safe state, restart the thread for that state
- Starvation – same thread may always be picked as victim, include number of rollback in cost factor

Deadlock – Management - Summary

Deadlock Prevention

- Invalidate any one condition.

Deadlock Avoidance

- RAG Scheme – when single Instance of Resource exists.
- Bankers Algorithm - when multiple Instance of Resource exists.

Deadlock Detection

- Wait for graph. - single Instance of Resource exists
- Detection algorithm using Available, Allocated, Requested, Need - multiple Instance of Resource exists

Deadlock Recovery

- Process Termination.
- Resource Preemption.